

- container is a far better choice in this case.
2. *You are more comfortable with Windows than Linux:*
There is no option in this case; you have to go with KVM because OpenVZ does not provide cross-platform virtualisation.
 3. *You are hosting a Java based application on a JBoss server:*
Go with KVM because of the extra load a Java application will create on the host machine. In OpenVZ, this load can be mistaken as a misuse or abuse by the provider, resulting in the suspension of your container.
 4. *You need a Linux machine for testing and need it quick:*
Go for OpenVZ as containers can be created within seconds. Later in this article you will see how this is possible.
 5. *You need a network device, or access to the kernel for adding/loading certain modules:*
Go with KVM, because it has its own dedicated kernel to which you will have access. You can load any module you like. OpenVZ is not an ideal choice in this case because if you want a certain module to be loaded, you will have to contact your provider, who will decide whether or not to load the specific module for you, depending upon various factors.
Everything depends on your needs, so choose wisely!

So how do you get very cheap servers these days?

This is one of the most frequently asked questions (FAQs) I get from the readers of my blog and the answer is: virtualisation. Instead of providing dedicated hardware to consumers, providers make use of one virtualisation technique or the other, and divide the same physical machine into numerous self-contained servers. Consumers can run various applications on these servers, such as a blog, purchase/lease these servers with shared resources at very low prices. This is a win-win situation for the consumer as well as the provider, in most cases. The consumer gets a dedicated and isolated server with full root user access, and the provider leases a part of the dedicated server at a low cost.

You might ask how this is different from normal Web hosting. Well, first of all, you do not get shell access to a shared hosting server, let alone the root level access. Second, the performance you get with Web hosting accounts ranks very low compared to what you get from a virtualised server.

A good place to start the search for cheap virtual servers is <http://lowendbox.com/>

Prerequisites

In this section, we will go through the prerequisites on the host system in order to run OpenVZ successfully and efficiently. In the following sections, we will get our host to run the OpenVZ kernel instead of the stock kernel, followed by some configuration changes and, finally, create a new virtual machine (container) along with some configuration.

I have jotted down some key requisites below:

1. A server based on Red Hat Enterprise Linux 6. This includes CentOS 6, RHEL 6, Scientific Linux 6 -- basically any Linux OS that is a derivative of RHEL 6 should work, although I have tested this set-up with RHEL 6 and CentOS 6 only.
2. Minimum 4GB RAM: I recommend this because the system itself needs to have some free RAM, let alone the VMs you will create within.
3. A 100GB hard disk.
4. SELinux set to 'Off'.

```
echo "SELINUX=disabled" > /etc/sysconfig/selinux
```

5. OpenVZ YUM repository installed.

```
# cd /etc/yum/repos.d/
# wget http://download.openvz.org/openvz.repo
# rpm --import http://download.openvz.org/RPM-GPG-Key-OpenVZ
# yum clean all
```

Installation

Installation of OpenVZ is facilitated by installing a patched kernel, which is provided by openvz.org. Run the following commands to get this set-up:

```
# yum list vzkernel
```

This will list all the available OpenVZ kernels from the repository. Choose the one that you want to install according to your server operating system architecture:

```
# yum install vzkernel -y
```

This should also update the GRUB (bootloader) entry, but I recommend you cross-check that the `vzkernel` is booted by default every time the system restarts. Open `/boot/grub/menu.lst` with your favourite text editor, and make sure the value of `default` is set to 0 and the title `OpenVZ (kernel-version)` is on the top of the various lists of available kernels. Refer to Figure 1.

If you are using a different bootloader (such as `syslinux`), then check the configuration file accordingly.

Before rebooting the host server, install a few other goodies that are needed in order to create/manage the containers:

```
# yum install vzctl vzquota ploop -y
```

Next, edit the `/etc/sysctl.conf` file and make sure you have the following parameters defined:

```
net.ipv4.ip_forward = 1
net.ipv4.conf.default.proxy_arp = 0
net.ipv4.conf.all.rp_filter = 1
kernel.sysrq = 1
```